Лекция 7. Деревья решений

Мы успели уже рассмотреть несколько алгоритмов из класса линейных моделей, каждая из которых подходит к определённой задаче по-своему. Однако у всех них есть один огромный недостаток: все они могли восстанавливать только линейные зависимости, т.е.

$$y = w_0 + w_1 x_1 + w_2 x_2 + \dots$$

а такое, на самом деле, далеко не всегда бывает в реальности. Если быть точнее, то со временем искомая зависимость в данных становится всё сложнее и сложнее, соответственно так просто воспользоваться линейными моделями мы уже не можем.

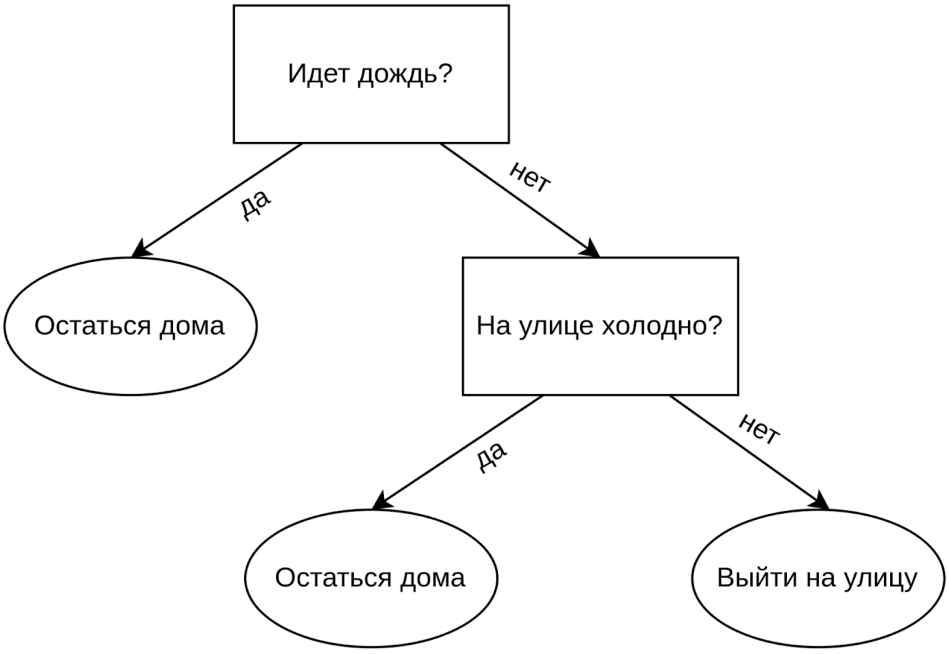
И тут на сцену выходит класс *нелинейных алгоритмов*. Их суть должна быть понятна из названия — они способны эффективно предсказывать данные, зависимость в которых представлена нелинейно.

Можно подумать, что всё, что мы изучали ранее — лишь моральная подготовка к нелинейным моделям, однако в ходе занятий мы также разберёмся, почему линейные модели вообще существуют, и неужели иногда лучше строить всё же их. Тем не менее, начиная с этого занятия, мы будем изучать только нелинейные модели, и объяснять, в чём заключается их магия.

Деревья решений

Итак, на повестке дня — *деревья решений* (Decision trees). Скорее всего, Вы уже прекрасно знаете, что это такое, просто не знали, как это называется. Довольно часто можно встретиться с картинками, которые объясняют их суть, да даже не это важно: размышления человека в некоторых ситуациях крайне похожи на то, что из себя представляют деревья решений.

Давайте взглянем на пример такого дерева:



Да, именно это из себя и представляет дерево решений! В данном случае, оно выполняет задачу классификации: на основе определённых факторов мы решаем, остаться дома или выйти на улицу. Причём просим обратить Ваше внимание на то, что необязательно опираться на какой-то один критерий: конкретно здесь применяется и дождь, и относительная температура. Мы бы в теории могли сюда ещё добавить, например, скорость ветра, время суток и т.д.

На самом деле, деревья решений подходят как для задач классификации, так и для задач регрессии. Поэтому, говоря более формально, мы будем изучать CART (Classification and Regression Tree).

Теперь, когда мы познакомились с тем, что из себя представляют CART, самое время дать им более формальное определение. Решающее дерево состоит из:

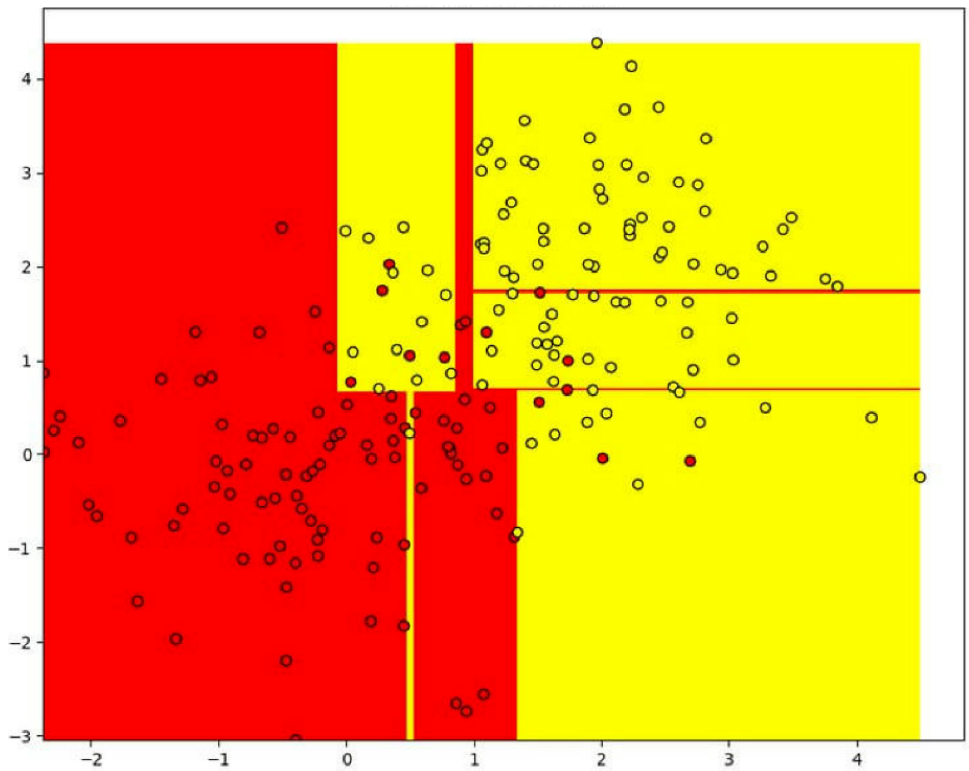
— Корневой узел;
— Ветви;
— Решающие (внутренние) узлы, листовые (терминальные) узлы.

Можно заметить, что это крайне похоже на бинарное дерево, и в общем случае Вы будете правы. Однако, деревья можно строить не обязательно бинарные, а значительно сложнее.

Итак, представьте что мы «льём» нашу выборку в это дерево сверху вниз. Корневой узел на пару с решающими узлами представляют собой некоторые «вопросы», в зависимости от ответа на которые мы будем разделять поток наших объектов направо и налево. Эти вопросы мы будем называть *предикатами*.

Так будет происходить до тех пор, пока объекты не достигнут терминальных узлов, которые уже будут совершать предсказание.

В случае с линейными моделями мы предполагали, что если построить модель для двумерного случая, то модель будет являться некоторой прямой. Давайте взглянем на то, как выглядит модель дерева решений:



Ну... давайте разбираться. Здесь изображена задача классификации, жёлтым — один класс, красным — другой класс. Дерево ставит своей задачей разбить пространство так, чтобы в каждой области лежали объекты только определённого класса. Соответствующим цветом выделена область, которая относится к классу такого же цвета (с точки зрения дерева решений). Каждый листовой узел соответствует определённой прямоугольной области на *графике границ решений*.

На самом деле, можно предположить, что с помощью дерева решений можно построить классификатор, который будет идеально разбивать *почти* любой случай. Это правда, однако это бессмысленно — дерево будет жутко переобученным, и от него будет мало толку. На рисунке в том числе можно заметить последствия переобучения: маленькие линии другого цвета среди другого класса свидетельствуют о том, что дерево намеренно построило разбиение так, чтобы захватить 1-2 объекта из этой области.

И тут особо остро стоит вопрос, который мы уже ранее обсуждали — выбросы в данных. Здесь они играют более губительную роль, и уже даже в процессе построения дерева может наделать делов и окончательно сломать дерево.

Подумайте: дерево может идеально разбить почти любой случай, это мы уже обсуждали. Однако не зря выделено слово почти — а какой случай дерево бы не смогло разбить? (Подсказка: тут явно замешаны вероятности).

Проблема градиентного спуска

Мы уже показали, что идеальные разбиения, зачастую, являются переобученными деревьями, но что если попытаться найти такое разбиение, которое

— Идеально классифицирует выборку;

— Не переобучено?

И тут ситуация, крайне похожая на линейные классификаторы: поиск такого разбиения можно осуществить только перебором, что опять является NP-полной задачей.

А теперь обсудим другой вопрос — а что там с градиентым спуском? На самом деле, не очень похоже на то, что тут можно как-то адекватно его применить. В первом приближении это действительно так, но стоит лишь взглянуть на задачу под другим углом.

Давайте построим следующий жадный алгоритм, который позволит с нуля строить деревья:

FitNode(m, RM):
  if выполнен критерий остановы: **then**
    находим cm
    объявляем вершину листовой
    return
  end

Критерий остановы (в контексте нашей лекции) — условие, при соблюдении которого мы прекращаем строить дерево, считая, что оптимальное уже достигнуто. Например:

— Если достигли максимальной глубины;

— Если количество объектов в узле меньше некоторого порога;

— ...

Если же критерий остановы не выполнен, то мы хотим получить какое-то разбиение для предиката. Будем рассматривать для предиката

$$[x_j < t]$$

Будем искать оптимальные порог и признаки такого предиката следующим образом:

$$j_*, t_* = \arg\max_{j,t} \; Q(R_m, j, t)$$

Появляется масса вопросов:

— Что такое Q? Как его искать?
— Как найти прогноз в листовой вершине?
— Какой критерий остановы подойдёт?
— Как находить оптимальные пороги и признаки?

Критерий остановы

Проще всего будет разобраться с критерием остановы. На самом деле, выше уже приведены такие критерии, которые подойдут. Можно и придумать свои, но главное, чтобы они были осмысленными.

Оптимальные пороги и признаки

Тут уже немного сложнее. Для начала важно понять суть предикатов: формально, мы задаём нашей выборке какой-то «вопрос», по результатам которого разделяем выборку на части. И текущая задача заключается в том, чтобы эти «вопросы» были как можно более эффективными. Но что значит эффективными?

Представьте, что Вы играете в игру "20 вопросов в которой один человек загадывает личность, а второй должен попытаться отгадать её, задавая только вопросы с ответами "Да"/"Нет". И здесь, аналогично, отгадывающий вынужден корректно подбирать вопросы. Например, если человек безуспешно сразу попытается отгадать, то у него останется ещё 8 миллиардов вариантов того, кто это может быть. Однако если спросить пол, то это отсечёт уже около половины кандидатов.

За этими рассуждениями стоит крайне важное понятие во многих технических сферах — *энтропия* (степень хаоса). Формально, она определяется как

$$S = -\sum_{i=1}^{N} p_i \log_2 p_i$$

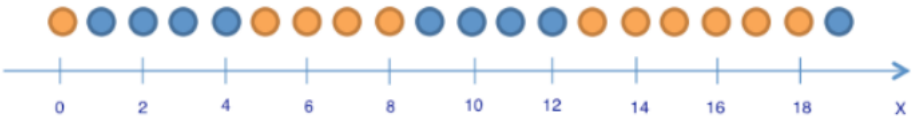
где p_i — вероятность появления i-го класса в рассматриваемом множестве, N — общее число классов. Используется соглашение о том, что $0 \log_2 0 = 0$.

Энтропия (Шеннона) принимает значения $[0, 1]$, где

— $0 \Rightarrow$ в узле наблюдается равномерное распределение (полный хаос, абсолютно случайные точки);

— $1 \Rightarrow$ вырожденное распределение (все объекты принадлежат одному классу, нет никакого хаоса).

Давайте перейдём к практическому примеру.



Здесь у нас есть 20 шариков, из которых 9 синих и 11 оранжевых. Соответсвенно,

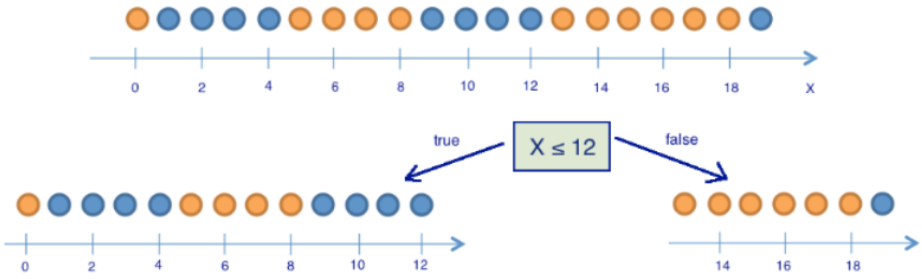
$$p_1 = \frac{9}{20}, p_2 = \frac{11}{20}$$

Посчитаем текущую энтропию:

$$S_0 = -\frac{9}{20} \log_2 \frac{9}{20} - \frac{11}{20} \log_2 \frac{11}{20} \approx 1$$

Энтропия даёт понять, что происходит примерно хаос — это можно заметить и визуально.

Давайте вручную сделаем предикат. Скажем, разобъём шарики на 2 лагеря по признаку $[x \leq 12]$:



В каждой из подвыборок посчитаем энтропию. Начнём с левой:

$$S_1 = -\frac{5}{13}\log_2\frac{5}{13} - \frac{8}{13}\log_2\frac{8}{13} \approx 0.96$$

А теперь для правой:

$$S_2 = -\frac{1}{7}\log_2\frac{1}{7} - \frac{6}{7}\log_2\frac{6}{7} \approx 0.6$$

Особенно для правого случая видим, что энтропия заметно снизилась, что заметно как и численно, так и визуально.

Формально, понижение энтропии ещё называют *приростом информации*, однако о подробностях этого процесса мы поговорим на следующем занятии.